

Pandas Tutorial 7: Data Cleaning and Advanced Operations

In this tutorial, students will learn how to clean data and work with string and datetime columns. Data cleaning is one of the most important parts of Exploratory Data Analysis.

1. Why Data Cleaning is Important

Real world datasets are not clean. They may contain:

- Missing values.
- Duplicate rows.
- Incorrect data types.
- Extra spaces in text.
- Wrong date formats.

Before performing analysis, we must clean the dataset properly.

Use these datasets in this lecture:

- `missing_values.csv`
 - `duplicates.csv`
 - `rename_columns.csv`
 - `string_cleaning.csv`
 - `datatype.csv`
 - `dates.csv`
 - `grades.csv`
 - `new_columns.csv`
-

2. Handling Missing Values

Use dataset:

```
pd.read_csv("missing_values.csv")
```

Check missing values:

```
df.isnull().sum()
```

Drop rows with missing values:

```
df.dropna()
```

Drop columns with missing values:

```
df.dropna(axis=1)
```

Fill missing values with specific value:

```
df.fillna(0)
```

Fill missing values with mean:

```
df["Age"] = df["Age"].fillna(df["Age"].mean())
```

3. Removing Duplicate Rows

Use dataset:

```
pd.read_csv("duplicates.csv")
```

Check duplicates:

```
df.duplicated().sum()
```

Remove duplicates:

```
df.drop_duplicates()
```

Duplicates can affect analysis results, so always check them.

4. Renaming Columns

Use dataset:

```
pd.read_csv("rename_columns.csv")
```

Sometimes column names are not clear.

Rename a column:

```
df.rename(columns={"old_name": "new_name"}, inplace=True)
```

Rename multiple columns:

```
df.rename(columns={  
    "old1": "new1",  
    "old2": "new2"  
}, inplace=True)
```

5. Working with String Data

Use dataset:

```
pd.read_csv("string_cleaning.csv")
```

Pandas provides string methods using `.str` accessor.

Convert to lowercase:

```
df["Name"] = df["Name"].str.lower()
```

Convert to uppercase:

```
df["Name"] = df["Name"].str.upper()
```

Remove extra spaces:

```
df["Name"] = df["Name"].str.strip()
```

Replace text:

```
df["City"] = df["City"].str.replace("isl", "Islamabad")
```

String cleaning is very common in EDA projects.

6. Changing Data Types

Use dataset:

```
pd.read_csv("datatype.csv")
```

Check data types:

```
df.dtypes
```

Convert column type:

```
df["Age"] = df["Age"].astype(int)
```

Convert to float:

```
df["Marks"] = df["Marks"].astype(float)
```

Correct data types are important for calculations.

7. Working with Date and Time

Use dataset:

```
pd.read_csv("dates.csv")
```

Convert column to datetime:

```
df["Date"] = pd.to_datetime(df["Date"])
```

Extract year:

```
df["Year"] = df["Date"].dt.year
```

Extract month:

```
df["Month"] = df["Date"].dt.month
```

Extract day:

```
df["Day"] = df["Date"].dt.day
```

Date handling is very important in time based analysis.

8. Applying Custom Functions

Use dataset:

```
pd.read_csv("grades.csv")
```

We can apply custom functions using `apply()`.

Example:

```
def grade(marks):  
    if marks >= 90:  
        return "A"  
    elif marks >= 80:  
        return "B"  
    else:  
        return "C"  
  
df["Grade"] = df["Marks"].apply(grade)
```

This creates a new column based on conditions.

9. Creating New Columns

Use dataset:

```
pd.read_csv("new_columns.csv")
```

We can create new columns using calculations.

Example:

```
df["Total"] = df["Marks1"] + df["Marks2"]
```

Or using condition:

```
df["Status"] = df["Marks"] > 50
```

Creating new features is called feature engineering.

10. Conclusion of Tutorial 7

In this tutorial, students have learned:

1. How to handle missing values.
2. How to remove duplicates.
3. How to rename columns.
4. How to clean string data.
5. How to change data types.
6. How to work with datetime columns.
7. How to apply custom functions.
8. How to create new columns.

Data cleaning and transformation are essential steps before visualization and machine learning.

This completes Pandas Tutorial 7.